

Sonification Summer Project Report

Daniel Hayton

July 2026

Contents

1	Introduction	2
2	Python Packages	3
3	Probability Branches	4
3.1	Curriculum Points Covered	4
3.2	Program Background	5
3.3	Sonification Implementation	6
4	Electrical Components	8
4.1	Curriculum Points Covered	8
4.2	Preliminaries and Mapping	9
4.3	Diode	13

4.4	Ohmic Resistor	15
4.5	Lamp	16
4.6	Future Developments	18
5	Interactive Velocity-Time Graph and Newton's Forces	19
5.1	Motive	19
5.2	Curriculum Points Covered	20
5.3	Style	21
5.4	General Mapping	22
5.5	Tempo Mapping	23
	5.5.1 Tempo Type 1: Frequency Map	23
	5.5.2 Tempo Type 2: Double Beat Map	24
	5.5.3 Tempo Type 3: Baseline Beat Map	24
5.6	Physics Implementation	25
5.7	User Controls	27
5.8	Sonification Generation	27
5.9	Future Developments	30
6	Next Steps	33

1 Introduction

This report will summarise the work I have produced as part of my summer project where I worked on creating sonifications using the STRAUSS Python package. The sonifications I produced were

aimed towards students studying KS3 maths and science with an emphasis on physics. I have produced three main programs out of the project each focusing on different subject areas and having sonifications implemented in different ways: live interactive velocity-time graph, parameterised current-voltage graphs for different electrical components and probability branches. I have also focused on tempo mapping which will be covered more in the velocity-time section and converted scripts from version 1 STRAUSS into version 1.5 STRAUSS for the Black Hole planetarium show; however, this was mainly a learning exercise for me so it will not be covered in detail in this report. All the Python code which I have written for this project can be found on my GitHub repository at <https://github.com/Daniel-Hayton/Sonification-Summer-Project>.

2 Python Packages

The main Python package I have used in this project is STRAUSS, as per the parameters and goals of the project, but I have also used several other packages that were vital to how my scripts run. These are Matplotlib, NumPy and Pygame. The first two played a very standard role, as is to be expected in most scientific Python code. I used Matplotlib for graph plotting to make and save visual figures and NumPy to perform calculations using NumPy arrays and functions.

Pygame was the most useful package other than STRAUSS in regards to sonification; this allowed me to save the .wav file with STRAUSS and then play it with Pygame. This was useful for having the version 1.5 STRAUSS code play in the console without requiring the user to select an audio file manually, but it was vital in my live graph otherwise the sonifications would not be able to play while the graph was still being altered. While the Pygame mixer was the primary tool used, other facets of Pygame were used more heavily in the live graph program and gTTS was also used for dynamic text-to-speech. These aspects will be covered in more detail later in the velocity-time section.

3 Probability Branches

3.1 Curriculum Points Covered

The points in the maths key stage curriculum which this program aims to cover are listed below [1].

- Probability
 - Pupils should be taught to:
 - * record, describe and analyse the frequency of outcomes of simple probability experiments involving randomness, fairness, equally and unequally likely out-

- comes, using appropriate language and the 0-1 probability scale
- * understand that the probabilities of all possible outcomes sum to 1
- * generate theoretical sample spaces for single and combined events with equally likely, mutually exclusive outcomes and use these to calculate theoretical probabilities.

3.2 Program Background

This is my smallest and most basic program. It attempts to demonstrate how quickly different branches can be created using simple, fair probability. The user is given the choice of a coin or an n -sided dice, for which the user can select the number of sides. Specifically, the coin example can be transferred from a purely mathematical context to a computer science background, as it demonstrates to students how a simple heads or tails, true or false, 1 or 0 can result in so many different outcomes, thereby linking to how computers are programmed.

The sonification is highly useful, as it can be difficult for those with limited vision to perceive an entire probability tree at once. The sonification works by generating the entire range of notes in the standard musical note set (A1 to G8). This range is then divided up by the number of branches that need to be played in that loop.

Each branch is given a distinct note with all the notes having equal separation. In addition to this, panning was used so that each note has its own unique stereo location. This program was also an opportunity to test these mapping techniques, which were used more intensely in my electrical components program.

3.3 Sonification Implementation

The thinking behind using both spatial and note separation was to give the user two avenues to differentiate the branches, which could easily become homogeneous if both steps were not taken. For example, if note separation was not used at high branch numbers, the pan separation would become indistinguishable from a single note. The panning also has the added benefit of making the user feel immersed in the sonification rather than it being something merely to be observed. This, I feel, will encourage the user to pay closer attention to what they are perceiving rather than giving the auditory equivalent of a quick glance.

An unexpected side effect of the note separation was that chords were produced naturally. I do not have enough musical knowledge to know exactly why this is the case, but they are distinctly present. It may be related to the particular mathematical structure of Western music, which could serve as another teaching point for the students.

The sonification of each branch was done using two nested for loops, with the outer `i` loop controlling how many branches there

were at that level and the inner `j` loop iterating through each branch. The style file would be rewritten in each `j` loop to change the note to be used and the `fix_pan` parameter would be calculated and set from the `j` value. Using Pygame gave me the ability to play the sonification of all the branches at that level at the end of the `i` loop. The program would wait using a while loop until the file had stopped playing and then the next level would start playing immediately afterwards, as it had already been processed and was ready to go. To avoid file overflow errors, I overwrote the same audio file each time, keeping the file storage concise and clean. This was done in a similar way to how the YAML style file was rewritten in each `i` loop.

Once the number of branches exceeded the number of available notes, I would allow this "all-note" branch sonification to play, but then the program would terminate afterwards. This prevents any errors arising from STRAUSS trying to sonify too many objects at once. The program does not break without this limit, but the gaps between sonifications become larger and larger as the required processing increases. I decided to allow the final all-note sonification to be played so that each run of the program results in the same sound, showing how the same result can be reached with different starting values. This, I feel, helps to convey the impression of infinite possibilities originating from a simple starting point. After this, a message is displayed that says "Too many notes to play", which gives the user confirmation that the program has stopped.

4 Electrical Components

4.1 Curriculum Points Covered

The points in the science key stage curriculum which this program aims to cover are listed below [2].

- Electricity and electromagnetism
 - Current electricity
 - * electric current, measured in amperes, in circuits, series and parallel circuits, currents add where branches meet and current as flow of charge
 - * potential difference, measured in volts, battery and bulb ratings; resistance, measured in ohms, as the ratio of potential difference (p.d.) to current
 - * differences in resistance between conducting and insulating components (quantitative).

The main learning objectives for this section are shown above. These give the requirement that students need to know and recognise current-voltage (IV) graphs for key electrical components. These components are the lamp, diode, resistor, LDR and thermistor. My program allows the user to set parameters to generate their own IV graph which has an accompanying sonification.

4.2 Preliminaries and Mapping

The user can decide how many components they want on their IV graph and select between a lamp, a diode and a resistor. These were the initial components chosen for the implementation as they do not depend upon external stimuli, unlike LDRs and thermistors which depend on light and temperature respectively. These extra dependencies made them difficult to integrate into the program at the initial stage; however, in its current form, there is definitely room for them to be included, which is a key point for future development.

The user will be asked generic questions at the start of each graph generation, such as the range of voltages they want. This range of voltages will be used throughout the graph generation and will be written into the YAML style file so that the time axis, which holds the voltage, can be scaled correctly. They will also be asked what kind of mapping they would like, with a choice between using a synthesizer or a song.

The main body of the program is in a **while True:** loop which means that it will repeat and be ready for users to make new graphs until the quit command is typed. The set up questions are asked in each loop and the previous loop has no effect on the current loop. Due to the large number of inputs that are taken the main body of the code is also encased in a **try except** statement which runs the code and if there is an error it will output a short error message to the user and start again.

The synthesizer mapping has the most parameters, utilising **volume** for resistance, **pitch_shift** for current and **volume_lfo** for temperature. It is useful to have these three parameters as separate mappings as it allows students to hear the relationship in the "behind-the-scenes" physics of the graphs. As an example, for a lamp, they can hear the pitch increase (representing the increase in current), which leads to the sonification getting louder (representing the resistance increasing) in response to the stutter in the sound (representing the temperature increasing). This helps to deliver the key concept that as current increases, temperature increases, which causes resistance to increase, thereby explaining the shape of the graph.

The song mapping (songification) is more basic as it does not involve the **volume_lfo** mapping. This was a conscious choice, as I thought that the beats in the song would mask the gaps left by the LFO. An alternative I had considered, but was not able to find the appropriate files to enact, was having the different layers of the song mapped to different quantities (e.g. the bassline and vocals being mapped to different parameters). This could provide an extra layer of interpretation; for instance, a graph showing the component in a steady state would have the tracks of the song line up to make the song sound normal.

In the actual implementation, the song used the same **pitch_shift** mapping for the current and the **volume** mapping for the resistance. The song chosen for this was "She's Electric" by Oasis, as it added a bit of humour and would hopefully increase engagement for those

listening. A royalty-free version of the song was used instead to avoid any concerns regarding piracy allegations.

The song had to be stored as a .wav file and placed in a folder to be used as a sampler. The song needed a file name that was a valid musical note to be recognised by STRAUSS. This meant it used 'sampler' generator type with it treated as a object source type, even though it was the only file in there. The pitch shift had the effect of slowing down and speeding up the track as well as altering its pitch. I believe this is due to how STRAUSS handles pitch shifting by increasing the speed of the track to make it reach a certain frequency or note.

The song was labelled as 'E5' as the song is in the key of E, though this label is arbitrary and it could have been labelled as any note. I used the standard two-octave pitch range, which meant that in order to have my song finish at regular speed and pitch, I had the start note set in the style file as 'E3'. Initially, I had accidentally forgotten to include the square brackets around 'E5' in the style file, which had the effect of returning the array for a chord. This made the song start low and finish higher than its actual speed. This code worked despite the sample not containing all the notes necessary for the chord.

When I realised this, I made the code more robust by changing it to only read for a single note and I reconstructed the sound by using ['E4'] in the style file. I ultimately decided to change it to ['E3'] so that the listener has more of a sense of resolution when

the sonification comes to an end. This is especially useful for the lamp, where the current plateaus to a top value, making it sensible for the listener to hear how the song is supposed to sound in case they are unfamiliar with it. This indicates that STRAUSS must be taking my sample and altering what it finds as an E5 note to generate an E4, which is a useful feature, though not one I was initially aware of.

The user has a choice of how they want each component to be identified in the audio plot. This lifts some of the behaviour from the probability branches section, with the user being able to choose between note separation, spatial separation, both or none. On the visual plot, the lines are, as usual, depicted in a different colour with a legend that includes both the name of the component and the resistance of that component, if applicable.

The spatial separation works on the same principle as the probability branches, but there is a difference in how the note separation works. In the electrical components program, a component is given a start note and an end note, which ensures that if this option is selected, there will be no overlap in frequencies. The end note of one component is given as the start note of the next component to be layered onto the figure. This option is, however, only available with the synthesizer style due to the extensive alterations to the notes required; if applied to the songification, it would prevent the track from finishing on its end note at the correct tempo. The note separation is useful as otherwise it can be hard to tell apart different components

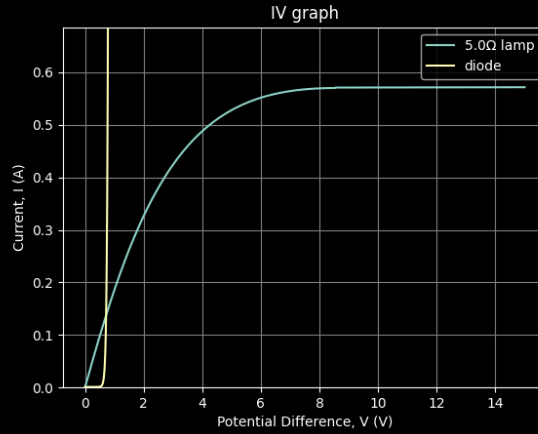


Figure 1: IV graph containing the plots for both a diode and a lamp based of the inputs minimum voltage $0V$, maximum voltage $15V$ and cold resistance 5Ω

and gives an option to those who may be hearing impaired and do not have sufficient hearing in both ears for the pan separation to be effective.

4.3 Diode

The diode selection has the least amount of alteration based on the user's input, with only the minimum voltage being taken into account directly. Other aspects of how the diode is visually and audibly plotted are influenced by any of the other components set up in the figure. The maximum voltage for the diode is capped at 1 V , as this gives a good estimate value of 0.7 V for the trigger voltage of a

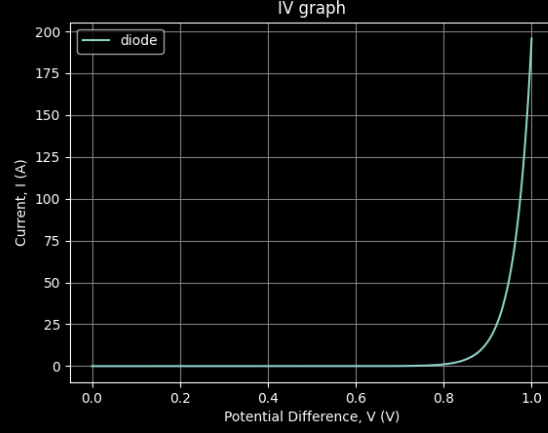


Figure 2: Diode graph produced when minimum voltage is given as 0

diode, allowing current to flow freely afterwards. The diode's graph is generated by using a piecewise equation (1) to encompass both the sharp exponential increase in current past the trigger voltage and the way in which the curve on the I-V graph goes practically vertical due to the resistance tending to zero. The diode graph was less about customisability and more about conveying the nature of a diode specifically, how it only allows current to flow in one direction and how resistance tends to zero after the trigger voltage, which is a very low value.

$$I(V) = \begin{cases} 10^{-3}, & V < 0, \\ I_s (e^{26V} - 1) + 10^{-3}, & V \geq 0. \end{cases} \quad (1)$$

Some parameters were preset in this program, as the knowledge of what would be acceptable values is beyond the Key Stage 3 (KS3) level; for example, the saturation current I_s is manually set to 10^{-9} A. The equation has an additive element of 10^{-3} as this makes the line more easily distinguished from the axis when plotting and it is true that only a perfect diode could block all the voltage across it.

The way in which the diode is affected by the other components is due to the fact that the graph must be clipped to have a sensible region for the plotting of the other components. This is because the current on the diode goes much higher than any other component. Consequently, the diode must be added as the final component when creating the figure so that the maximum and minimum values of the standard range are known. The input was specifically validated to avoid having the diode enter as a not the final component. A point of future development for this component is to allow the user to decide which direction they want the diode to act in.

4.4 Ohmic Resistor

The Ohmic resistor is the simplest component as it is governed by a simple linear equation (2). The user has the added choice of the resistance they want the resistor to have, which is then represented as the reciprocal of the gradient on the plot. The `volume_lfo` is kept at zero as there is no change in the resistance and the students only need to be acquainted with a perfect Ohmic conductor. However,

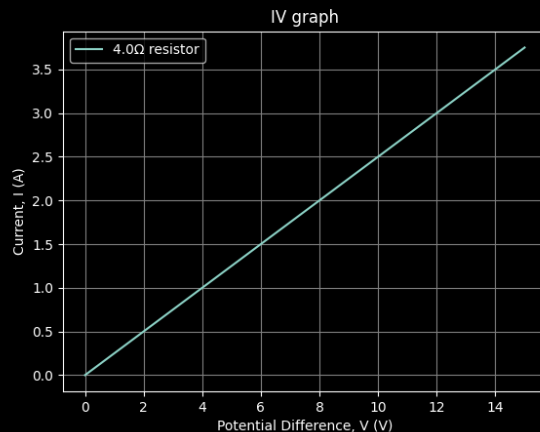


Figure 3: Resistance graph produced with inputs minimum voltage $0V$, maximum voltage $15V$ and resistance 4Ω

there is room to add an option for the user to select a non-perfect conductor which has temperature dependencies, although this is already somewhat covered by the lamp option.

$$V = IR \quad (2)$$

4.5 Lamp

From the user's point of view, the lamp behaves very similarly to the resistor, asking the user to input a level of cold resistance. The term "cold resistance" would likely be unfamiliar to a KS3 student, but I included it so that they can have some customisability over the lamp they select. Equally, this element complements the resistor

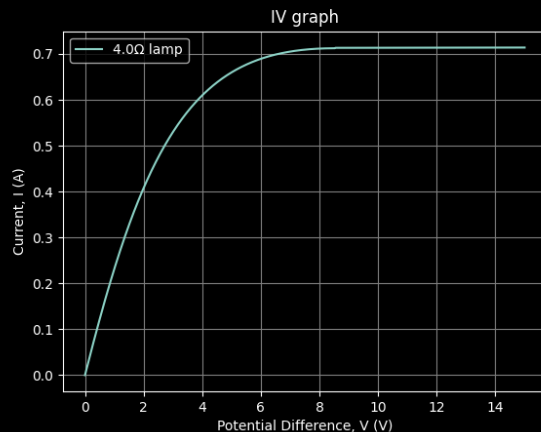


Figure 4: Lamp graph produced with inputs minimum voltage $0V$, maximum voltage $15V$ and cold resistance 4Ω

section and demonstrates how a higher resistance behaves, showing that different lamps have different filaments and therefore different resistances.

Other values used in the governing equation that are too advanced for KS3 are the temperature coefficient α and the curve factor n , which I have set to $\alpha = 0.08$ and $n = 1.5$ to give the most realistic, generic results. One main physical governing equation was used as the basis for the line generation, as shown in (3); however, this was only suitable for low voltages, so it had to be altered by the use of a piecewise function, as shown in equation (4). This adjustment prevents the current from straying into a region that implies a decrease in resistance, making it clear to students that a higher current results in a higher resistance.

$$I_0(V) = \frac{V}{R_0 (1 + \alpha|V|^n)}, \quad (3)$$

$$I(V) = \begin{cases} I_0(V_{\min}) + 10^{-4}V, & V < V_{\min}, \\ I_0(V), & V_{\min} \leq V \leq V_{\max}, \\ I_0(V_{\max}) + 10^{-4}V, & V > V_{\max}. \end{cases} \quad (4)$$

Unlike the previous two examples, the temperature had to be calculated to be mapped onto the `volume_lfo` parameter. This was done through the use of equation (5), where α is the same preset value as before, T_0 is room temperature (293 K), R_0 is their inputted cold resistance and $R(T)$ was calculated at each point using Ohm's law.

$$T = T_0 + \frac{(R(T)/R_0) - 1}{\alpha}, \quad (5)$$

4.6 Future Developments

Due to the fact that this program uses a text-based interface running in the console, inputting parameters can become repetitive and boring for users. This is why a key point for future development would be to implement a GUI (Graphical User Interface) so that it is quicker for the user to change parameters and observe changes more immediately.

Another key point for future development could be the way in which the behaviour of a graph holding multiple components is treated. At the moment, the graph generation treats them all individually, as though the data were taken from a test circuit with one component being changed out to be examined. This is a desirable feature; however, it could be improved by giving the user the option to generate the graph with the components being in isolation (status quo), in series or in parallel. This would help cover more points on the curriculum and could show how the construction of circuits can affect current behaviour, not just resistance and voltage.

5 Interactive Velocity-Time Graph and Newton's Forces

5.1 Motive

This program was created as a proof of concept for live sonifications to allow for a highly interactive and immersive experience for the listener. This approach means that they are actively interacting with the data rather than being a passive, third-party observer. In a certain regard, they are creating their own sonification, which makes them more likely to consider making their own sonifications in the future since they have already done so here. A live graph was chosen as this provides users with an easily interpretable visual output along-

side the audio output. A velocity-time graph was selected because it is a key part of the KS3 curriculum and the graph can be modified by changing the force acting upon the object being plotted, which directly relates to Newton's laws another key learning objective.

5.2 Curriculum Points Covered

The points in the science key stage curriculum which this program aims to cover are listed below [2].

- **Describing motion:**

- Speed and the quantitative relationship between average speed, distance and time (speed = distance $\div$ time).
- The representation of a journey on a distance-time graph.

- **Forces:**

- Forces as pushes or pulls, arising from the interaction between two objects.
- Using force arrows in diagrams, adding forces in one dimension and balanced versus unbalanced forces.
- Forces associated with deforming objects (stretching and squashing springs), friction between surfaces and resistance to motion in air and water.

- How forces are measured.
- **Balanced forces:**
 - Opposing forces and equilibrium: weight held by a stretched spring or supported on a compressed surface.
- **Forces and motion:**
 - Forces being needed to cause objects to stop or start moving, or to change their speed or direction of motion (qualitative only).
 - Changes depending on direction of force and its size.

5.3 Style

When designing the program, I wanted to make the sonification relate to the real world using sounds that users are accustomed to, in order to give a sense of familiarity and intuition when listening to it for the first time. Equally, I wanted the physics in the graph to be accurate to a real-world object. When performing the conversions for the Black Hole show, I discovered that the white noise generator can be used to sound very similar to a steam train. Consequently, this is what I decided to base my velocity-time graph around, with the user effectively controlling the train by basic means such as increasing

velocity by increasing the driving force, or bringing it to a halt using the braking force.

5.4 General Mapping

The sound profiles and mappings were designed to align with the overarching theme of a steam train. Additionally, I wanted to aim for each sound to have its own distinct mapping to increase clarity and ensure the user does not get confused. The mapping techniques I used are shown in Table 1.

Parameter	Mapping	Emulates
Driving force	<code>volume_lfo</code>	Chuffing
Brake force	<code>pitch_shift</code>	Squeaking brakes
Friction	<code>cutoff</code>	Wind noise
Velocity	Tempo mapping	Clickety-clack
Balanced net force	Fixed note	Rail rumble
Local maximum	Volume	High whistle
Local minimum	Volume	Low whistle

Table 1: The mappings used for the interactive velocity-time graph and their intended resemblances

5.5 Tempo Mapping

In the audio experiment folder, I carried out most of my investigations into tempo mapping. I attempted three main types: one in which the basic frequency of clicks was mapped to the magnitude of a curve; a second which used a double beat, with the spacing between the pairs being mapped to the magnitude and the spacing between the two beats; and a third type with a standard beat and a second beat which would be closer or further away to map the magnitude.

5.5.1 Tempo Type 1: Frequency Map

This type was simple and was heavily based on James Trayford's code, with the basic integration and interpolation of each the data points mapped using time events that would signify the height of the graph. This first method worked well and I was initially going to use this in my train mapping, with the velocity represented by puffs being a time event. However, it is more basic and I decided that the driving force should be represented by the chuffs, as that implies the actual work done by the engine rather than the resulting velocity. However, this method does not lend itself well to abrupt changes in the values of the forces. This is what caused me to change the train mapping from a tempo style to a `volume_lfo` style which still uses rhythmic elements.

5.5.2 Tempo Type 2: Double Beat Map

The double beat mapping with an added layer of complexity was my intended mapping for the velocity utilising a clickety-clack style. The two-beat method did work for a simple graph, but the second beat does not help to convey any more information and that is why I wanted to extend this by having it store the information of the acceleration as well. The tempo mapping implementation does not seem suited to having two separate sonifications, as they can easily fall out of sync. This is also due to how STRAUSS removes blank space to ensure the best use of the data as possible. When attempting to use features in the style files such as **input_range**, the timings did not seem to be corrected or altered to fit a set window of the length of the sonification. This is a main issue with the tempo mapping, as it can require a long sonification and takes as long as it needs for that particular set of data. I had tried to normalise the inputs, but the style of mapping did not work well in a standard temporal shape.

5.5.3 Tempo Type 3: Baseline Beat Map

This method of implementation had a few issues. One of these was the fact that, in order for the second beat to sound before the baseline beat for the next data point played, the gap between the baseline beats had to be separated by a large time spacing. This could make the sonification quite long and could make it harder for the listener

to remember the data points and build an idea of the overall data in their head. It was important that the second beat did not bleed into the data area for the following data point, as this would effectively turn it into the frequency mapping option with a regular timekeeping beat in the background. This is not an entirely unwanted form of sonification as a mapping using tempo type 1 there can be large sections of silence, which can lead to listener auditory disorientation. If there was a beat to help them keep time the sections of silence would be avoided and the listener would have a good sense of where they are in the sonification; however, I feel it would be better if the regular ticks could also convey some form of encoded data.

5.6 Physics Implementation

The physics used to program this simulation relied by its very nature on basic equations for velocity and forces. Newton's first law (conceptually implemented) was used to calculate the net force acting on the train, as shown in equation (7) and Newton's second law, equation (8), was used to calculate the acceleration experienced by the train. This acceleration was then fed into the SUVAT equation, equation (6), to calculate the velocity coordinate to be plotted on the graph.

$$v = u + at \tag{6}$$

$$F_n = F_D + F_B + F_f \quad (7)$$

$$F = ma \quad (8)$$

If friction was active in the program, equation (9) was used to calculate its value. Equation (9) utilizes the directly proportional relationship between the square of the velocity and the fluid resistance experienced by a moving object. This is an abstracted form of the drag equation, serving primarily as a physical mechanism to prevent the velocity from escalating off the top of the graph.

The factor σ acts as a speed limit scaling factor. I calibrated the equation such that a driving force of 200 kN produces a terminal speed of 50 m s⁻¹, which can be altered by changing the value of σ from its default of 1.

$$F_f = -80 \operatorname{sgn}(v) \left(\frac{v}{\sigma} \right)^2 \quad (9)$$

There are a few extra edge cases where custom logic had to be implemented in the code because the analytical equations could not govern every scenario, but these remain the primary guiding physical principles.

5.7 User Controls

The user mainly interacts with the program by changing the forces using the arrow keys. Due to the large emulated mass decided for this program $m = 10^5$ kg, the forces had to increase and decrease in steps of 10 kN in order for the key presses to make a noticeable difference. This could have been avoided by using a smaller simulated mass, but I felt it was important to keep the attributes of the train realistic.

Other key commands were included, including those for spoken readouts of the current parameters. The spoken readouts are important to ensure that blind and low-vision users can know exactly what forces are acting on the train and what velocity has resulted from them, without having to rely solely on estimating from the sonification. The program used Google's Text-to-Speech (gTTS) library; however, other Python text-to-speech packages would work suitably in its place. The full list of keyboard commands is shown in Table 2.

5.8 Sonification Generation

The way in which the sonification behaves is not as originally intended. This is because the graph is not truly live and there is a 10 second delay from when the forces have affected the graph to this being reflected in the sonification. The original implementation used the current force parameters set by the user and projected what the sonification would be over time using these parameters. This meant

Key Press	Action
q	Quits the program
Up Arrow	Increases driving force by 10 kN
Down Arrow	Decreases driving force by 10 kN
Right Arrow	Increases braking force by 10 kN
Left Arrow	Decreases braking force by 10 kN
r	Resets driving and braking forces to zero
Left Shift	Resets the driving force to 0
Right Shift	Resets the braking force to 0
Tab	Toggles friction
Spacebar	Toggles sonification
Backspace	Toggles the sign of the driving force
s	Speaks all current data on the screen
d	Speaks the driving force
b	Speaks the braking force
n	Speaks the net force
f	Speaks the frictional force
v	Speaks the current velocity

Table 2: Keyboard commands available during the running of the program

that the sonification would only reload if a user pressed a key or if the sonification ran until the end of its duration. This worked in the very initial stages of development; however, when the complexity of

the sonification grew, it became apparent that the implementation was not suitable. This was because if a user made a lot of changes in quick succession, a sonification would be generated each time and when the sonification had more layers this generation would take longer. This made the graph not as interactive an experience, as the user would have to wait for the graph to load frequently and the computer had the capacity to terminate the program if it tried to generate too many sonifications in too quick a succession. Additionally, when friction was introduced this made performing the projection calculation even more complex and caused me to switch to the second type of implementation.

The current form of implementation instead has the data sonified as it leaves the screen on the left, rather than as it appears on the screen on the right. This initially had the same problems with generation time, despite the fact that the issue of constant generation whenever a key was pressed had been avoided. It was impossible for the computer to generate the sonification instantaneously, which would cause the sonification to stop and fall out of sync with the graph.

The way in which this problem was resolved was by placing an imaginary divider down the centre of the graph. The sonification of the left-hand side would be playing whilst the sonification for the right-hand side was being generated. This meant that the new sonification to be played on the left was compiled and ready for when Pygame had finished playing the previous sonified section of

the graph. This kept the sonification in time and reduced the pauses in the sonification to a small blip when the tracks changed. This did mean that twice as many sonifications were being produced, but they contained half the data, so the overhead was minimal. It did mean that the program required a starting sonification to play; otherwise, it would play the last section of the graph from the previous time the program was run on start-up. This was avoided by flushing the data in the sonification file by writing over it with a sonification of zeros, matching what is displayed on the screen. Due to this sonification needing to be finished before a new one could be generated and played, the controls of the graph are unavailable until the initial two sonifications have ended and the time range has been reached.

In order to make sure that the sonification sounded consistent between the sonified sections of the graph, the **input_range** feature was used to make sure that STRAUSS would produce the sonification in accordance with the largest and smallest possible values for the sonified parameters within the program. This ensured consistency when listening to different sections and meant that they could be compared with one another.

5.9 Future Developments

There are several directions in which this program can be taken in the future. One such route would be a form of gamification, which would provide the user with clear goals and objectives to give them

a purpose for using the program. For example, a goal could involve navigating a certain route from station to station, or achieving specific graph shapes. This would encourage the user to engage with the program longer, moving away from their current method of interaction, which is largely limited to aimless experimentation and fiddling. The mechanics of such a game could also convey the physics content more deeply and encourage the user to think about how the forces directly affect the motion displayed on the graph. Some potential goals could include:

- Turn off friction and keep the train at a constant velocity (which could be made more challenging by specifying a target velocity).
- Drive the train for a set period of time, ensuring that the total displacement is zero by the end of the time limit.
- Create consecutive turning points close together to generate a two-tone whistle.

Another area for future development is the tempo mapping for the velocity, as the current implementation does not accurately reflect the authentic "clickety-clack" train sound. The ultimate intention is that the time interval between the clicks signifies the magnitude of the velocity; the closer together the clicks, the higher the velocity. In addition to this, the time interval between the "clicks" and the

”clacks” was intended to be linked to the acceleration. Changes in the applied forces would alter the spacing between the ”clickety” and the ”clack,” adding an extra layer of data encoding. This could also make the difference between acceleration and deceleration much clearer; the sonification could be designed to play ”clack-click” when decelerating, rather than the standard ”click-clack.”

A small point for future development would be the ability to activate and deactivate each sonification element individually. As it stands, the user can only turn the entire sonification on or off. However, users could benefit from some extra control in the sound options, as some students could find all of the sounds overwhelming, while other students may struggle to hear certain elements if they are masked by other aspects of the sonification. Equally, it may be useful if a teacher is using the program to cover a certain point with a specific set of features on the graph. The ability to turn off the other, non-relevant elements of the sonification for the demonstration would be useful so that students can hone in on the key factors at play.

Another way in which the program could be developed would be to link it to a physical, real-world element to introduce a tactile aspect. This could involve linking the program to a small model train, where the voltage applied to the track is directly related to the net force calculated in the program. This physical coupling would help students properly conceptualise how a vehicle behaves under these forces and how the velocity-time graph maps to reality. This

could be achieved using a microcomputer such as a Raspberry Pi and would serve to deeply engage students by offering an unconventional departure from their normal classroom experience.

6 Next Steps

The first major next step would be to have trials of the sonifications created with test groups in order to gain responses on people's opinions of the sonifications and sound profiles, to measure their usefulness as a teaching aid and how pleasant they are to interact with. These 'audiotors' would be able to provide feedback that could alter the program's mid-development, following a RAD style of development technique.

Depending on the response from the 'audiotors', the suggested forms of future development from this report could be implemented or, alternatively, different ideas could be introduced and incorporated from the stakeholder feedback. This cycle could be repeated a few times, or possibly a more waterfall approach could be taken in order to produce a more clearly structured program.

A final route for future development for all the programs detailed in this report would be to release them as stand-alone executable files or embed them into a wider, classroom-focused sonification suite. This would make the software highly accessible to educators and researchers, providing an opportunity to test the reception and effec-

tiveness of these programs in a live classroom setting.

References

- [1] Department for Education, “Mathematics programmes of study: Key stage 3 (National Curriculum in England),” Department for Education, UK Government, National Curriculum Framework DFE-00179-2013, Sep. 2013. [Online]. Available: <https://www.gov.uk/government/collections/national-curriculum>
- [2] —, “Science programmes of study: Key stage 3 (National Curriculum in England),” Department for Education, UK Government, National Curriculum Framework DFE-00185-2013, Sep. 2013. [Online]. Available: <https://www.gov.uk/government/collections/national-curriculum>